

430.211 Programming Methodology (프로그래밍 방법론)

Taesup Moon

Department of Electrical and Computer Engineering
Seoul National University

Outline

- Chapter 2: Flow of Control

Chapter 2: Flow of Control

:What are we learning?

- **Boolean Expressions**
 - Building, Evaluating & Precedence Rules
- **Branching Mechanisms**
 - `if-else`
 - `switch`
 - **Nesting** `if-else`
- **Loops**
 - `while`, `do-while`, `for`
 - **Nesting** loops
- **Introduction to File Input**

Branching Mechanisms

: Example

- `if-else` statements

- Choice of two alternate statements based on condition expression

- Example:

```
if (hrs > 40)
```

Check whether
True or False



```
    grossPay = rate*40 + 1.5*rate*(hrs-40);
```

```
else
```

```
    grossPay = rate*hrs;
```

- Most branching mechanisms are controlled by Boolean expressions

Boolean Expressions

- Logical operators
 - Logical AND (&&)

```
(2 < x) && (x < 7)
```

- Logical OR (||)

```
(y < 0) || (y < 12)
```

Boolean Expressions

- Comparison operators

Display 2.1 Comparison Operators

MATH SYMBOL	ENGLISH	C++ NOTATION	C++ SAMPLE	MATH EQUIVALENT
=	Equal to	==	<code>x + 7 == 2*y</code>	$x + 7 = 2y$
≠	Not equal to	!=	<code>ans != 'n'</code>	$ans \neq 'n'$
<	Less than	<	<code>count < m + 3</code>	$count < m + 3$
≤	Less than or equal to	<=	<code>time <= limit</code>	$time \leq limit$
>	Greater than	>	<code>time > limit</code>	$time > limit$
≥	Greater than or equal to	>=	<code>age >= 21</code>	$age \geq 21$

Evaluating Boolean Expressions

- Variable type, `bool`
 - Can store either of the values `True` or `False`

```
bool result = (x < z) && (z < y);
```

- Boolean expression can be evaluated in the same way that an arithmetic expression is evaluated
- E.g., let `y=8` . What is

```
!( (y < 3) || (y > 7) ) ?
```

Evaluating Boolean Expressions: : Truth Tables

Display 2.2 Truth Tables

AND

<i>Exp_1</i>	<i>Exp_2</i>	<i>Exp_1 && Exp_2</i>
true	true	true
true	false	false
false	true	false
false	false	false

OR

<i>Exp_1</i>	<i>Exp_2</i>	<i>Exp_1 Exp_2</i>
true	true	true
true	false	true
false	true	true
false	false	false

NOT

<i>Exp</i>	<i>!(Exp)</i>
true	false
false	true

Evaluating Boolean Expressions: : Precedence Rules (I of 4)

Display 2.3 Precedence of Operators

::	Scope resolution operator
.	Dot operator
->	Member selection
[]	Array indexing
()	Function call
++	Postfix increment operator (placed after the variable)
--	Postfix decrement operator (placed after the variable)
++	Prefix increment operator (placed before the variable)
--	Prefix decrement operator (placed before the variable)
!	Not
-	Unary minus
+	Unary plus
*	Dereference
&	Address of
new	Create (allocate memory)
delete	Destroy (deallocate)
delete[]	Destroy array (deallocate)
sizeof	Size of object
()	Type cast

*Highest precedence
(done first)*

Evaluating Boolean Expressions: : Precedence Rules (2 of 4)

* / %	Multiply Divide Remainder (modulo)
+ -	Addition Subtraction
<< >>	Insertion operator (console output) Extraction operator (console input)



*Lower precedence
(done later)*

Evaluating Boolean Expressions: : Precedence Rules (3 of 4)

Display 2.3 Precedence of Operators

All operators in part 2 are of lower precedence than those in part 1.

<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal
!=	Not equal
&&	And
	Or

Evaluating Boolean Expressions: : Precedence Rules (4 of 4)

=	Assignment
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign
%=	Modulo and assign
? :	Conditional operator
throw	Throw an exception
,	Comma operator



*Lowest precedence
(done last)*

Evaluating Boolean Expressions : Examples

- How to evaluate the following?

$$x + 1 > 2 \quad || \quad x + 1 < -3$$

- Roughly, arithmetic done before the comparison & logical
- Followings are all equivalent

$$(x + 1) > 2 \quad || \quad (x + 1) < -3$$
$$((x + 1) > 2) \quad || \quad ((x + 1) < -3)$$

- **Short-circuit** evaluation

- Evaluation stops when the result is already decided

- E.g., $(x \geq 0) \ \&\& \ (y > 1)$

- if $x < 0$, the expression is definitely `False` since $(x \geq 0)$ is `False` → evaluation stops!

Evaluating Boolean Expressions : Examples

- Short-circuit evaluation
 - Be careful with increment operators!
 - E.g., `(x > 1) && (y++)`
 - If `x > 1` is `False`, `y++` is not executed!
 - Avoid using `++` or `--` operator within `&&` or `||`
- Integers as Boolean values
 - All non-zero values $\rightarrow$ `True`
 - Zero value $\rightarrow$ `False`

Branching Mechanism

:if-else statement syntax

- Formal syntax:

```
if (<boolean_expression>)  
    <yes_statement>  
else  
    <no_statement>
```

- Note each alternative is only **ONE** statement.
- To have multiple statements execute in either branch
→ Use **compound** statement

if-else Branching Mechanism : Compound/Block statements

- Must use compound statement `{ }` for multiple statements
 - Also called as a "block" statement
 - Can be used even if there is just one statement
 - Enhances readability

if-else Branching Mechanism : Compound/Block statements in Action

- Note indenting in this example:

```
if (myScore > yourScore)
{
    cout << "I win!\n";
    wager = wager + 100;
}
else
{
    cout << "I wish these were golf scores.\n";
    wager = 0;
}
```

Common Pitfalls!

:if-else

- Using = in place of ==
 - The former means "assignment" (=)
 - The latter means "equality" (==)
 - They have VERY different meanings in C++!
 - E.g., :

```
if (x = 12)
    Do_Something
else
    Do_Something_Else
```

Wrong!

if-else Branching Mechanism

: Omitting the else

- else clause is **optional**

- If, in the false branch, you want "nothing" to happen, leave it empty

- E.g.,

```
if (sales >= minimum)
    salary = salary + bonus;
cout << "salary = $" << salary;
```

- If False, nothing happens

- Execution continues with `cout` statement

if-else Branching Mechanism : Nested Statements

- if-else statements contain smaller statements
 - Compound or simple statements (we've seen)
 - Can also contain any statement, including another if-else statement!
 - Example:

```
if (speed > 110)
    if (speed > 130)
        cout << "You're really speeding!";
    else
        cout << "You're speeding.";
```

- Note proper indenting!

if-else Branching Mechanism : Multiway if-else

- Multiway if-else statement is just another variation
 - In order to prevent excessive indentation
 - E.g.,

```
if ((temperature < -10) && (day == SUNDAY))  
    cout << "Stay home.";  
else if (temperature < -10) // and day != SUNDAY  
    cout << "Stay home, but call work.";  
else if (temperature <= 0) // and temperature >= -10  
    cout << "Dress warm.";  
else // temperature > 0  
    cout << "Work hard and play hard.";
```

The `switch` Statement

- Controls multiple branches
 - Can do the same thing with `if-else`, but sometimes `switch` is more convenient
- Uses a controlling expression which returns either
 - `Bool`
 - `Enum` (will be discussed later)
 - Integer type
 - Character

The switch statement syntax

switch Statement

SYNTAX

```
switch (Controlling_Expression)
{
    case Constant_1:
        Statement_Sequence_1
        break;
    case Constant_2:
        Statement_Sequence_2
        break;
        .
        .
        .
    case Constant_n:
        Statement_Sequence_n
        break;
    default:
        Default_Statement_Sequence
}
```

You need not place a break statement in each case. If you omit a break, that case continues until a break (or the end of the switch statement) is reached.

The switch statement syntax

EXAMPLE

```
int vehicleClass;
double toll;
cout << "Enter vehicle class: ";
cin >> vehicleClass;

switch (vehicleClass)
{
    case 1:
        cout << "Passenger car.";
        toll = 0.50;
        break;
    case 2:
        cout << "Bus.";
        toll = 1.50;
        break;
    case 3:
        cout << "Truck.";
        toll = 2.00;
        break;
    default:
        cout << "Unknown vehicle class!";
}
```

If you forget this break, then passenger cars will pay \$ 1.50.

The switch statement syntax

- `break`
 - `switch` ends when `break` is encountered or the end is reached
- We can have two cases run the same code

```
case 'A':  
case 'a':  
    cout << "Excellent. "  
        << "You need not take the final.\n";  
    break;
```

- Since no `break` statement in the first case
- `default` : when no cases are matched
 - Good to include (not mandatory)

Common pitfalls / tips!

:switch

- Forgetting the `break`
 - No compiler error
 - Execution simply "falls thru" other cases until reaching `break`;
- `if-else` is more versatile, but sometimes `switch` is more handy
 - E.g., implementing MENUs
 - Provides clearer "big-picture" view
 - Shows menu structure effectively
 - Each branch is one menu choice

switch Menu example

```
switch (response)
{
    case 1:
        // Execute menu option 1
        break;
    case 2:
        // Execute menu option 2
        break;
    case 3:
        // Execute menu option 3
        break;
    default:
        cout << "Please enter valid response.";
}
```

enumeration types

- enumeration type
 - Defined by a list of constants of type `int`
 - E.g., following defines constants for each month

```
enum MonthLength { JAN_LENGTH = 31, FEB_LENGTH = 28,  
    MAR_LENGTH = 31, APR_LENGTH = 30, MAY_LENGTH = 31,  
    JUN_LENGTH = 30, JUL_LENGTH = 31, AUG_LENGTH = 31,  
    SEP_LENGTH = 30, OCT_LENGTH = 31, NOV_LENGTH = 30,  
    DEC_LENGTH = 31 };
```

- When no values are assigned, consecutive values starting from 0 are assigned

```
enum Direction { NORTH = 0, SOUTH = 1, EAST = 2, WEST = 3 };  
  
enum Direction { NORTH, SOUTH, EAST, WEST };
```

Equivalent!

```
enum MyEnum { ONE = 17, TWO, THREE, FOUR = -3, FIVE }; ??
```

Conditional Operator

- Also called "ternary operator"
 - Allows embedded conditional in expression
 - Essentially "shorthand if-else" operator
 - Example:

```
if (n1 > n2)
    max = n1;
else
    max = n2;
```

- Can be written as :

```
max = (n1 > n2) ? n1 : n2;
```

- "?" and ":" form this "ternary" operator

switch / enum 예시

```
#include <iostream>
using namespace std;

int main()
{
    enum Direction {NORTH, SOUTH, EAST, WEST};
    int dir_input;
    cout << "Input your direction: " << endl;
    cin >> dir_input;

    switch (dir_input)
    {
        case NORTH:
            cout << "Your direction is NORTH" << endl;
            break;
        case SOUTH:
            cout << "Your direction is SOUTH" << endl;
            break;
        case EAST:
            cout << "Your direction is EAST" << endl;
            break;
        case WEST:
            cout << "Your direction is WEST" << endl;
            break;
        default:
            cout << "invalid direction" << endl;
    }

    return 0;
}
```

- switch의 control_expression에는 변수명이 들어감
- dir_input에 integer를 입력하게 됨.
- ⇔
- switch(Direction) 꼴은 **error**
- 단, 아래와 같이는 가능!

```
Direction dd;
dd=WEST;

switch (dd)
{
    . . .
}
```

Three Types of Loops in C++

- `while`
 - Most flexible
 - No "restrictions"
- `do-while`
 - Least flexible
 - Always executes loop body at least once
- `for`
 - Natural "counting" loop

while Loops Syntax

A **while** STATEMENT WITH A SINGLE-STATEMENT BODY

```
while (Boolean_Expression)  
    Statement
```

A **while** STATEMENT WITH A MULTISTATEMENT BODY

```
while (Boolean_Expression)  
{  
    Statement_1  
    Statement_2  
    .  
    .  
    .  
    Statement_Last  
}
```


while Loop Example

```
1  #include <iostream>
2  using namespace std;

3  int main( )

4  {
5      int countDown;

6      cout << "How many greetings do you want? ";
7      cin >> countDown;

8      while (countDown > 0)
9      {
10         cout << "Hello ";
11         countDown = countDown - 1;
12     }

13     cout << endl;
14     cout << "That's all!\n";

15     return 0;
16 }
```

Sample Dialogue 1

How many greetings do you want? 3
Hello Hello Hello
That's all!

Sample Dialogue 2

How many greetings do you want? 0
That's all!

*The loop body is executed
zero times*

do-while Loops Syntax

A do-while STATEMENT WITH A SINGLE-STATEMENT BODY

```
do  
    Statement  
while (Boolean_Expression);
```

A do-while STATEMENT WITH A MULTISTATEMENT BODY

```
do  
{  
    Statement_1  
    Statement_2  
    .  
    .  
    .  
    Statement_Last  
} while (Boolean_Expression);
```

*Do not forget
the final
semicolon.*

do-while Loop Example

```
1  #include <iostream>
2  using namespace std;

3  int main( )
4  {
5      int countDown;

6      cout << "How many greetings do you want? ";
7      cin >> countDown;

8      do
9      {
10         cout << "Hello ";
11         countDown = countDown - 1;
12     } while (countDown > 0);

13     cout << endl;
14     cout << "That's all!\n";

15     return 0;
16 }
```

Sample Dialogue 1

```
How many greetings do you want? 3
Hello Hello Hello
That's all!
```

Sample Dialogue 2

```
How many greetings do you want? 0
Hello
That's all!
```

*The loop body is always
executed at least once.*

while vs. do-while

- Very similar, but a subtle difference
 - One difference regarding checking Boolean expression
 - `while` : checks BEFORE the body is executed
 - `do-while`: checked AFTER the body is executed
- What's the point of iterating zero times (as in `while`)?
 - Ex1) when summing the numbers in a list, it can be empty!
 - A checkbook balancing program
- `while` is more common, due to its "flexibility"

Increment/decrement operators

- Generally discouraged to use in the Boolean expressions → But, if done with cares, it can work out.

```
1  #include <iostream>
2  using namespace std;

3  int main( )
4  {
5      int numberOfItems, count,
6          caloriesForItem, totalCalories;

7      cout << "How many items did you eat today? ";
8      cin >> numberOfItems;
9      totalCalories = 0;
10     count = 1;
11     cout << "Enter the number of calories in each of the\n"
12          << numberOfItems << " items eaten:\n";

13     while (count++ <= numberOfItems)
14     {
15         cin >> caloriesForItem;
16         totalCalories = totalCalories
17             + caloriesForItem;
18     }

19     cout << "Total calories eaten today = "
20          << totalCalories << endl;
21     return 0;
22 }
```

Sample Dialogue

```
How many items did you eat today? 7
Enter the number of calories in each of the
7 items eaten:
300 60 1200 600 150 1 120
Total calories eaten today = 2431
```

What is the value of count after printing the totalCalories?

for Loop Syntax

for Statement Syntax

```
for (Initialization_Action; Boolean_Expression; Update_Action)  
    Body_Statement
```

No semicolon!

- A general looping mechanism that can do anything that a `while` can do
 - Like `if-else`, *Body_Statement* can be a block statement
 - The third *Update_Action* is optional
 - Often used to step through an array (Ch.5)

for Loop Syntax

No semicolon!

for Statement Syntax

```
for (Initialization_Action; Boolean_Expression; Update_Action)  
    Body_Statement
```

- Followings are all equivalent!
(comma operator allows multiple actions!)

```
sum = 0;  
int n;  
for (n = 1; n <= 10; n++)  
    sum = sum + n;
```

```
for (sum = 0, n = 1; n <= 10; n++)  
    sum = sum + n;
```

```
for (sum = 0, n = 1; n <= 10; sum = sum + n, n++);
```

```
for (sum=0, n=1;n<=10;){  
    sum+=n;  
    n++;  
}
```

for Loop Syntax

EXAMPLE

```
for (number = 100; number >= 0; number--)  
    cout << number  
        << " bottles of beer on the shelf.\n";
```

- How many times does the loop body executes?

–

– Initialization, loop condition, update all “*built in*”

– Execution order:

*Initialization → check condition → loop body → update action →
check condition → loop body → ...*

- **Q) Is it possible to have a for loop whose body is executed zero times?**

for Loop Syntax

EXAMPLE

```
for (number = 100; number >= 0; number--)  
    cout << number  
        << " bottles of beer on the shelf.\n";
```

EQUIVALENT while LOOP SYNTAX

```
Initialization_Action;  
while (Boolean_Expression)  
{  
    Body_Statement  
    Update_Action;  
}
```

EQUIVALENT EXAMPLE

```
number = 100;  
while (number >= 0)  
{  
    cout << number  
        << " bottles of beer on the shelf.\n";  
    number--;  
}
```

Sample Dialogue

```
100 bottles of beer on the shelf.  
99 bottles of beer on the shelf.  
.  
.  
.  
0 bottles of beer on the shelf.
```

Loop Issues

- Loop's condition expression can be ANY Boolean expression
- Examples:

```
while (count<3 && done!=0)
{
    // Do something
}
```

```
for (index=0; index<10 && entry!=-99)
//update is omitted
{
    // Do something
}
```

Loop Pitfalls : Misplaced ;

- Watch the misplaced ; (semicolon)

- Example:

```
for (int count = 1; count <= 10; count++);  
    cout << "Hello\n";
```

- Notice the ‘;’ after the `for` loop
- ; also is an **empty statement!**
- It will print ‘Hello’ only **once!**
- Nothing happens for

```
for (int count = 1; count <= 10; count++);
```

- Similar issue exists for the `while` loop

Loop Pitfalls : Infinite Loops

- Loop condition must evaluate to False at some iteration through the loop. If not, we have an **infinite loop**!

```
x = 2;  
while (x != 12)  
{  
    cout << x << endl;  
    x = x + 2;  
}
```

- Above is fine. But, if the initialization is $x=1$, it never ends!
- A safer choice: replace the Boolean expression with `while (x < 12)`

- Infinite loops can be desirable sometimes
 - e.g., airline reservation system

The break and continue statements

- Flow of Control
 - Recall how loops provide "graceful" and clear flow of control in and out
 - In RARE instances, we can alter natural flow
- `break;`
 - Ends the entire loop
- `continue;`
 - Ends the current iteration of the loop body
- These statements violate natural flow
 - Only used when absolutely necessary!

The break and continue statements

Display 2.8 A **break** Statement in a Loop

```
1  #include <iostream>
2  using namespace std;

3  int main( )
4  {
5      int number, sum = 0, count = 0;
6      cout << "Enter 4 negative numbers:\n";

7      while (++count <= 4)
8      {
9          cin >> number;

10         if (number >= 0)
11         {
12             cout << "ERROR: positive number"
13                 << " or zero was entered as the\n"
14                 << count << "th number! Input ends "
15                 << "with the " << count << "th number.\n"
16                 << count << "th number was not added in.\n";
17             break;
18         }

19         sum = sum + number;
20     }

21     cout << sum << " is the sum of the first "
22         << (count - 1) << " numbers.\n";

23     return 0;
24 }
```

Sample Dialogue

```
Enter 4 negative numbers:
-1 -2 -3 4
ERROR: positive number or zero was entered as the
4th number! Input ends with the 4th number.
4th number was not added in
-6 is the sum of the first 3 numbers.
```

The break and continue statements

Display 2.9 A continue Statement in a Loop

```
1  #include <iostream>
2  using namespace std;

3  int main( )
4  {
5      int number, sum = 0, count = 0;
6      cout << "Enter 4 negative numbers, ONE PER LINE:\n";

7      while (count < 4)
8      {
9          cin >> number;

10         if (number >= 0)
11         {
12             cout << "ERROR: positive number (or zero)!\n"
13                 << "Reenter that number and continue:\n";
14             continue;
15         }
16         sum = sum + number;
17         count++;
18     }

19     cout << sum << " is the sum of the "
20         << count << " numbers.\n";
21     return 0;
22 }
```

Sample Dialogue

```
Enter 4 negative numbers, ONE PER LINE:
1
ERROR: positive number (or zero)!
Reenter that number and continue:
-1
-2
3
ERROR: positive number!
Reenter that number and continue:
-3
-4
-10 is the sum of the 4 numbers.
```

Notice also the difference
in the **update statements**!

Nested Loops

- ANY valid C++ statements can be inside body of loop
- This includes additional loop statements!
 - Called "nested loops"
- Requires careful indenting:

```
for (outer=0; outer<5; outer++)  
    for (inner=7; inner>2; inner--)  
        cout << outer << inner;
```

- Notice no { } since each body is one statement

Introduction to File Input

- We can use `cin` to read from a file in a manner very similar to reading from the keyboard
- Only an intro is given here, more details in Ch. 12
 - Just enough so you can read from text files and process larger amounts of data that would be too much work to type in

Opening a Text File

- Add at the top

```
#include <fstream>
using namespace std
```

- Then, **declare an input stream** just as you would declare any other variable

```
ifstream inputStream;
```

- Next, connect the `inputStream` variable to a text file on the disk

```
inputStream.open("filename.txt");
```

- “**filename.txt**” denotes the the file is in the same directory as the code. (Need a pathname if the file is in other directory.))

Reading from a Text File

- Use

```
istream >> var;
```

- The result is the same as using `cin >> var` except the input is coming from the text file and not the keyboard
- Reading a string is done UP TO a whitespace character

- When done with the file close it with

```
istream.close();
```

- Releases any resources that have been allocated in association with the file

File Input Example (1 of 2)

- Consider a text file named `player.txt`

Display 2.10 Sample Text File, `player.txt`, to Store a Player's High Score and Name

```
100510  
Gordon Freeman
```

File Input Example (2 of 2)

- Consider a text file named `player.txt`

Display 2.11 Program to Read the Text File in Display 2.10

```
1  #include <iostream>
2  #include <fstream>
3  #include <string>

4  using namespace std;
5  int main( )
6  {
7      string firstName, lastName;
8      int score;
9      fstream inputStream;

10     inputStream.open("player.txt");

11     inputStream >> score;
12     inputStream >> firstName >> lastName;

13     cout << "Name: " << firstName << " "
14          << lastName << endl;
15     cout << "Score: " << score << endl;
16     inputStream.close();

17     return 0;
18 }
```

`ifstream`: input only
`ofstream`: output only
`fstream`: both input/output

Sample Dialogue

Name: Gordon Freeman
Score: 100510

Using a Loop to Read the Text File Input

- Consider a text file named `player.txt`

Display 2.12 Using a Loop to Read the Text File in Display 2.10 (part 2 of 2)

```
6 {
7     string text;
8     fstream inputStream;
9
10    inputStream.open("player.txt");
11
12    while (inputStream >> text)
13    {
14        cout << text << endl;
15    }
16    inputStream.close();
17
18    return 0;
19 }
```

*Evaluates to false
when there is no data left
to read from the file*

Sample Dialogue

```
100510
Gordon
Freeman
```

Performs two actions

- A read action
- Checking a Boolean result

Summary I

- Boolean expressions
 - Similar to arithmetic → results in True or False
- C++ branching statements
 - `if-else`, `switch`
 - `switch` statement great for menus
- C++ loop statements
 - `while`
 - `do-while`
 - `for`

Summary 2

- `do-while` loops
 - Always execute their loop body at least once
- `for` loop
 - A natural "counting" loop
- Loops can be exited early
 - `break` statement
 - `continue` statement
 - Usage restricted for style purposes
- Reading from a text file is similar to reading from `cin`

Thank you!

Web: <http://mindlab-snu.github.io>

E-mail: tsmoon@snu.ac.kr